Artist Revenue Management System

Database Design & Implementation

MDL018 – Data Organization and Management

Member A *ID_A*
Member B *ID_B*
Member C *ID_C*
Member D *ID_D*

Faculty of Information Technology
University of Science – VNU-HCM

April 4, 2026

Table of Contents

Introduction

Artist Revenue Management System

Objectives · Users · Technology Stack

System Objectives

Problem Statement

Build a **V-Pop music royalty & revenue management** system covering the full artist lifecycle: contracts, music catalog, multi-source revenue streams, and transparent financial settlement via digital wallets.

Three main revenue streams:

- **Streaming** — Spotify, Apple Music, Zing MP3
- **Sync Licensing** — films, ads, games
- **Live Performance** — concerts & shows

Core Requirements

- Contract management & revenue splitting by percentage
- Digital wallets + financial audit trail
- Real-time analytics dashboard
- Automated ETL pipeline from multiple data sources
- Data integrity (append-only revenue logs)

User Personas

Artist

View personal revenue, track albums/tracks, check wallet balance, view event schedule, request withdrawals

Record Label

Manage signed artists, create & manage contracts, distribute revenue via contract splits, monitor revenue logs

Manager / Booking Agent

Manage performance schedules, book venues, track event status

System Administrator

System-wide statistics, consolidated revenue reports, analytics dashboard, wallet audit

Technology Stack & Actual Data

Component	Technology
Database	Neon Serverless PostgreSQL 16
Stored Proc	8 SQL Functions (plpgsql)
Frontend	Streamlit + Plotly
ETL	Python + SQLAlchemy
Data Sources	MusicBrainz API + Ticketbox
VCS	Git (feat/fix → develop → main)

Loaded data (post-ETL):

Entity	Records
Artists	40
Albums	781
Tracks	250
Revenue Logs	3,180
Contracts	45
Contract Splits	498
Events	81
Artist Wallets	22

Conceptual Design (ERD)

Database Design

19 Tables · 3 ISA Relationships · Polymorphic FK

Schema Overview — 19 Tables, 4 Groups

Group 1 — Core Entities (7 tables)

`labels`, `artists` *(parent)*
`solo_artists`, `bands`, `composers` *(ISA children)*
`albums`, `tracks`

Group 2 — Contracts & Splits (5 tables)

`contracts` *(parent)*
`recording_contracts`, `distribution_contracts`, `publishing_contracts` *(ISA children)*
`contract_splits` *(polymorphic FK)*

Group 3 — Revenue (6 tables)

`revenue_logs` *(parent, append-only, partitioned)*
`streaming_revenue_details`
`sync_revenue_details`
`live_revenue_details` *(ISA children)*
`artist_wallets`, `withdrawals`

Group 4 — Events & Venues (3 tables)

`events`, `venues`, `managers`

Total: 19 tables (13 core tables + 6 ISA child tables)

ERD Diagram

Figure: Entity-Relationship Diagram — Artist Revenue Management System

ISA Relationships — 3 Inheritance Hierarchies

Artists ISA

Overlapping, Partial

An artist may hold **multiple roles simultaneously** or belong to **none** (e.g. unclassified DJ)

↓ `solo_artists`
↓ `bands`
↓ `composers`

Contracts ISA

Disjoint, Total

Every contract **must** belong to exactly one type — no overlap, no exceptions

↓ `recording_contracts`
↓ `distribution_contracts`
↓ `publishing_contracts`

Revenue Logs ISA

Disjoint, Total

Every revenue entry belongs to exactly one source — mandatory classification

↓ `streaming_revenue_details`
↓ `sync_revenue_details`
↓ `live_revenue_details`

Integrity Constraints & Business Rules

Key technical constraints:

- `tracks.isrc` — `UNIQUE NOT NULL VARCHAR(12)`, international standard tracking code
- `contracts.contract_id` — `UUID` PK, prevents ID enumeration attacks
- `artist_wallets.balance` — `CHECK >= 0`, balance never goes negative
- `revenue_logs` — **Append-only**, no `UPDATE` or `DELETE`
- `contract_splits` — `CHECK XOR(artist_id, label_id)`, exactly one beneficiary
- `bands.member_count` — `CHECK >= 2`

Business rules:

- **BR-01:** Total `share_percentage` per (contract, track) $\leq$ 1.0 (100%)
- **BR-02:** Wallet balance only decreases when withdrawal `status = 'completed'`
- **BR-03:** Revenue logs are append-only — corrections use a negative-amount reversal entry
- **BR-05:** Live revenue **must** link to a valid `event_id`

Implementation

System Implementation

Physical Design · ETL Pipeline · Indexes & Views

Physical Design — Highlights

Listing: artists — JSONB metadata

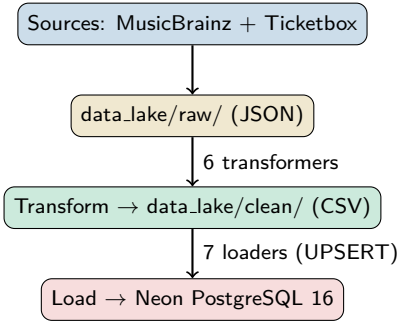
```sql
CREATE TABLE artists (
  artist_id   SERIAL PRIMARY KEY,
  stage_name  VARCHAR(100) UNIQUE NOT NULL,
  full_name   VARCHAR(200),
  debut_date  DATE,
  birthday    DATE,
  label_id    INT REFERENCES labels,
  metadata    JSONB  -- genre, social_links
);
```

Listing: revenue_logs — partitioned

```sql
CREATE TABLE revenue_logs (
  log_id       BIGSERIAL,
  track_id     INT REFERENCES tracks,
  amount       DECIMAL(15,4) NOT NULL,
  currency     VARCHAR(3) DEFAULT 'VND',
  log_date     DATE NOT NULL,
  revenue_type VARCHAR(20),
  raw_data     JSONB,
  PRIMARY KEY (log_id, log_date)
) PARTITION BY RANGE (log_date);
```

Quarterly partitions: `revenue_logs_2025_q1`, `revenue_logs_2025_q2`, ...

ETL Pipeline



Loaded records (post-ETL):

Table	Records
labels	183
artists	40
albums	781
tracks	250
events	81
revenue_logs	3,180
contracts	45
contract_splits	498
artist_wallets	22
withdrawals	19

Indexes, Views & Materialized Views

Key indexes:

- `idx_artists_metadata_gin` — **GIN** on `artists.metadata` JSONB, supports `@>` containment operator
- `idx_revlog_track_date` — Composite B-Tree (`track_id`, `log_date`), enables partition pruning
- `idx_splits_contract_track` — Composite for contract_splits JOIN
- `idx_contracts_active` — **Partial** index (`status = 'active'`)
- `idx_artists_name_lower` — **Expression** index on `LOWER(stage_name)`, supports case-insensitive ILIKE search

Views (real-time):

- `v_artist_revenue_summary`
- `v_contract_split_detail`

Materialized Views (cached):

- `mv_artist_revenue_summary`
- `mv_top_tracks_cached`
- `mv_monthly_revenue_by_source`
- `mv_contract_split_summary`

Refresh all: `CALL sp_refresh_all_mv();`

Data Exploitation

Data Exploitation

8 Stored Procedures · Advanced SQL Techniques

Overview — 8 Stored Procedures

#	Function Name	SQL Technique	Purpose
SP1	sp_revenue_by_artist_rollup	GROUP BY ROLLUP	Revenue by artist & month with subtotals
SP2	sp_revenue_pivot_by_source_v2	SUM() FILTER (PIVOT)	Pivot revenue by source type
SP3	sp_top_earning_artists	Nested subquery in HAVING	Artists above average revenue
SP4	sp_contract_revenue_distribution	LEFT JOIN LATERAL	Contract payout per beneficiary
SP5	sp_top_tracks_per_artist	RANK() OVER (PARTITION BY)	Top N tracks per artist
SP6	sp_wallet_audit_report	Correlated subqueries + JSONB	Wallet vs. earned reconciliation
SP7	sp_venue_event_analytics	CTE + ROLLUP + Window	Event & venue statistics
SP8	sp_search_artists	GIN index + JSONB @>	Artist search by metadata

SP1 — ROLLUP: Revenue by Artist & Month

Listing: sp_revenue_by_artist_rollup

```sql
SELECT
  COALESCE(a.stage_name,
    '*** GRAND TOTAL ***')::VARCHAR AS nghe_si,
  COALESCE(
    TO_CHAR(r.log_date, 'MM/YYYY'),
    'Full year')::TEXT              AS thang,
  COUNT(*)::BIGINT                  AS so_giao_dich,
  SUM(r.amount)                     AS tong_doanhthu
FROM revenue_logs r
JOIN tracks t  ON r.track_id = t.track_id
JOIN albums al ON t.album_id = al.album_id
JOIN artists a ON al.artist_id = a.artist_id
WHERE r.log_date >= make_date(p_year, 1, 1)
  AND r.log_date <  make_date(p_year + 1, 1, 1)
GROUP BY ROLLUP(
  a.stage_name,
  TO_CHAR(r.log_date, 'MM/YYYY'))
ORDER BY a.stage_name NULLS LAST;
```

Technique

- `ROLLUP(artist, month)` automatically generates 3 aggregation levels: per-month, per-artist subtotal, grand total
- `COALESCE(..., '*** GRAND TOTAL ***')` identifies summary rows
- Partition pruning on `revenue_logs` via `log_date`

Output levels

- Per artist + month
- Subtotal per artist
- Grand total (all artists)

SP5 — Window Function: Top Tracks per Artist

Listing: sp_top_tracks_per_artist

```
SELECT ranked.*
FROM (
  SELECT
    a.stage_name::VARCHAR        AS nghe_si,
    t.title::VARCHAR             AS track_title,
    al.title::VARCHAR            AS album_title,
    t.isrc::VARCHAR,
    t.play_count,
    COALESCE(SUM(r.amount), 0)   AS tong_doanhthu,
    -- Window: independent rank per artist
    RANK() OVER (
      PARTITION BY a.artist_id
      ORDER BY COALESCE(SUM(r.amount),0) DESC
    )                            AS hang
  FROM artists a
  JOIN albums al ON a.artist_id = al.artist_id
  JOIN tracks t  ON al.album_id = t.album_id
  LEFT JOIN revenue_logs r ON t.track_id = r.track_id
    AND r.log_date >= make_date(p_year, 1, 1)
    AND r.log_date <  make_date(p_year + 1, 1, 1)
  GROUP BY a.artist_id, a.stage_name,
           t.track_id, t.title, al.title,
           t.isrc, t.play_count
```

Technique

- `RANK() OVER (PARTITION BY artist_id ORDER BY revenue DESC)` — independent ranking per artist
- Subquery in `FROM` computes ranks before filtering
- `WHERE hang <= p_top_n` retrieves top N per artist

Example call

```
SELECT * FROM
sp_top_tracks_per_artist(3,
2025);
```

→ Top 3 tracks by revenue for each artist in

SP4 & SP6 — LATERAL JOIN & Wallet Audit

SP4: LATERAL JOIN

```sql
-- Compute track revenue once, reuse twice
LEFT JOIN LATERAL (
  SELECT COALESCE(SUM(r.amount), 0)
    AS total_rev
  FROM revenue_logs r
  WHERE r.track_id = t.track_id
) tr ON TRUE

-- Polymorphic beneficiary via ISA tables
LEFT JOIN artist_beneficiaries ab
  ON ab.beneficiary_id = cs.beneficiary_id
LEFT JOIN artists a_ben
  ON a_ben.artist_id = ab.artist_id
LEFT JOIN labels l_ben
  ON l_ben.label_id = lb.label_id
```

SP6: JSONB + Wallet Reconciliation

```sql
-- GIN index: extract genre from JSONB
(a.metadata->>'genre')::TEXT AS genre,

-- Audit check: balance vs earnings
CASE WHEN ABS(
  total_earned
  - total_withdrawn
  - wallet_balance
) < 0.01
  THEN 'OK'
  ELSE 'WARNING'
END AS audit_status
```

Result: 22 wallets audited
All differences ≈ 0
$\rightarrow$ audit_status = **OK**

SP8 — GIN Index + JSONB: Artist Search

Listing: sp_search_artists

```
SELECT
  a.artist_id,
  a.stage_name::VARCHAR,
  (a.metadata->>'genre')::TEXT      AS genre,
  (a.metadata->'social_links')::JSONB
                                    AS social_links,
  l.name::VARCHAR                   AS label_name,
  (SELECT COUNT(*)::BIGINT
   FROM tracks t JOIN albums al ON ...
   WHERE al.artist_id = a.artist_id
  ) AS total_tracks
FROM artists a
LEFT JOIN labels l ON a.label_id = l.label_id
WHERE
  -- idx_artists_metadata_gin (GIN @> operator)
  (p_genre IS NULL OR a.metadata @>
    jsonb_build_object('genre', p_genre))
  -- idx_artists_name_lower (expression index)
  AND (p_name IS NULL OR
    LOWER(a.stage_name) LIKE
    '%' || LOWER(p_name) || '%')
ORDER BY a.stage_name;
```

Technique

- `metadata @> jsonb_build_object('genre', p_genre)` — GIN containment, leverages `idx_artists_metadata_gin`
- `LOWER(stage_name) LIKE` — case-insensitive search via expression index
- Returns `social_links` as JSONB (instagram, tiktok, youtube)

JSONB metadata example

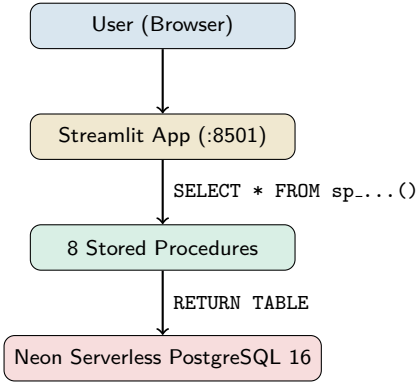
```
{"genre":  "V-Pop",
"social_links":  {
 "instagram":  "...",
```

Application Demo

Application Demo

Streamlit Dashboard · 5 Pages · 8 Stored Procedures

System Architecture



5 dashboard pages:

1. **Dashboard** — KPIs + revenue overview
2. **Artists** — Search + top earners *(SP8, SP3)*
3. **Revenue Analytics** — ROLLUP, PIVOT, Top tracks *(SP1, SP2, SP5)*
4. **Finance** — Contract splits + wallet audit *(SP4, SP6)*
5. **Events & Venues** — Event statistics *(SP7)*

Dashboard — Live Data (2025)

Figure: Dashboard page — Revenue Overview

KPIs from loaded data:

Artists 40	Rev. Logs 3,180
Tracks 250	Contracts 45
Wallets 22	Events 81

Revenue Analytics & Finance Pages

Revenue Analytics

Figure: SP1 ROLLUP + SP2 PIVOT + SP5 Top Tracks

Stacked bar: Streaming / Sync / Live per artist

Finance

Figure: SP6 Wallet Audit — balance vs. earnings

Audit status: OK / WARNING per artist

Conclusion

Conclusion

Summary · Technical Highlights · Team Contributions

Summary

Completed

- **19 tables** with 3 full ISA hierarchies (overlapping/partial + 2× disjoint/total)
- **8 Stored Procedures** — ROLLUP, PIVOT (FILTER), Nested subquery in HAVING, LATERAL JOIN, Window (RANK), Correlated subqueries, CTE + ROLLUP, GIN + JSONB
- **Automated ETL pipeline**: MusicBrainz + Ticketbox → Transform → Neon (3,180 revenue logs)
- **5-page Streamlit dashboard** backed by all 8 SPs
- Materialized Views + GIN / B-Tree / Partial / Expression indexes

Technical Highlights

- **Polymorphic FK** — `contract_splits` → Artist XOR Label (ISA beneficiaries table)
- **UUID PK** for contracts — prevents ID enumeration
- **Append-only** `revenue_logs` partitioned by quarter — immutable financial audit trail
- **JSONB metadata** + GIN index — flexible genre & social-link search
- **Wallet reconciliation** — difference ≈ 0 for all 22 artists (audit_status = OK)
- **Neon Serverless** — SSL enforced, SCRAM-SHA-256, connection pooling

Team Contributions

	Member	Responsibilities	Share
A	Member A	Requirements spec, ERD, logical design, report	25%
B	Member B	DDL scripts, seed data, views, indexes	25%
C	Member C	Stored Procedures (8 SPs), ETL pipeline	25%
D	Member D	Streamlit dashboard (5 pages), slides	25%

Git Workflow

`feat/fix/docs` branches
→ `develop` → `main`

Commit Convention

```
feat(db):  add ISA tables
fix(sp):  cast play_count to BIGINT
docs(erd):  update 19-table diagram
```

Thank you for your attention!

Q & A

We are ready to answer your questions.

Source Code & Database:

`https://github.com/<org>/artist-revenue-management`